

DISTRIBUTED LARGE SYSTEMS

Exam Project, EASV PB Software 2025

Synopsis

Architecting for Scalability and Resilience:

Klaus Hviid

31-05-2025



Distributed Large Systems

Contents

INTRODUCTION	3
1. PRINCIPLES FOR LARGE SYSTEMS.....	3
LOOSE COUPLING:.....	3
HIGH COHESION:	3
SCALABILITY:	3
RESILIENCE & FAULT ISOLATION:	3
DESIGN FOR FAILURE (CONCEPTUAL & IMPLEMENTED BASICS):	3
AUTOMATION:	4
2. OPTIMIZATION	5
CACHING WITH REDIS:	5
ASYNCHRONOUS PROCESSING:	5
DATABASE INDEXING:	5
CQRS (COMMAND QUERY RESPONSIBILITY SEGREGATION):	5
POLYGLOT PERSISTENCE:	5
3. DESIGN TO BE MONITORED	6
STRUCTURED LOGGING WITH SERILOG:.....	6
HEALTH CHECK ENDPOINTS:	6
DISTRIBUTED TRACING (PLANNED - OPENTELEMETRY):	6
METRICS (PLANNED - OPENTELEMETRY):	6
4. SCALABILITY CUBE	7
X-AXIS (HORIZONTAL DUPLICATION/CLONING):	7
Y-AXIS (FUNCTIONAL DECOMPOSITION/SERVICES):.....	7
Z-AXIS (DATA PARTITIONING/SHARDING):	7
DEMONSTRATE SCALING ON TWO AXES:	7
5. ASYNCHRONOUS DESIGN.....	8
ASYNCHRONOUS.....	8
CONTRAST WITH SYNCHRONOUS:.....	8
TIGHT COUPLING BETWEEN THE SERVICES.	8
6. TOOLS FOR LARGE SYSTEMS (PICK ONE NOT USED IN CLASS).....	9
YARP (API GATEWAY):.....	9
MASSTRANSIT (MESSAGING ABSTRACTION):	9
DOCKER COMPOSE (DEVELOPMENT ORCHESTRATION):	9
7. FAULT-ISOLATIVE ARCHITECTURES	10
MICROSERVICE BOUNDARIES:.....	10
ASYNCHRONOUS COMMUNICATION VIA RABBITMQ:	10
API GATEWAY (YARP) WITH HEALTH CHECKS:.....	10
DATABASE PER SERVICE:	10
(CONCEPTUAL/PLANNED) CIRCUIT BREAKER PATTERN (POLLY):.....	10
8. SPECIFICATIONS AND OUTSOURCING	11
API CONTRACTS (OPENAPI/SWAGGER):.....	11
EVENT CONTRACTS (SYNOPSIS/SL.SHARED.EVENTS):	11

APPLICATION LAYER INTERFACES:	11
HOW TO USE FOR OUTSOURCING:	11
9. FEATURE FLAGS.....	12
DEMONSTRATE (CONCEPTUALLY):	12
PROS & CONS:	12
10. AVAILABILITY	13
MICROSERVICE INDEPENDENCE & REDUNDANCY (X-AXIS SCALING):	13
ASYNCHRONOUS COMMUNICATION VIA RABBITMQ:	13
API GATEWAY HEALTH CHECKS:.....	13
FAULT ISOLATION BY DESIGN:	13
HOW YOU COULD POTENTIALLY HARM AVAILABILITY:.....	13
11. DEVOPS.....	14
AUTOMATION:	14
TESTING (PLANNED):	14
DEPLOYMENT (CONCEPTUAL FOR CI/CD):.....	14
RECOVERY:.....	14
12. GREEN SOFTWARE	15
RESOURCE EFFICIENCY OF MICROSERVICES (Y-AXIS & X-AXIS SCALING):	15
OPTIMIZED DATABASE CHOICE (POLYGLOT PERSISTENCE):	15
ASYNCHRONOUS PROCESSING & EVENT-DRIVEN ARCHITECTURE:	15
CACHING (REDIS):	15
DEMONSTRATION (CONCEPTUAL COMPARISON FOR SYNOPSISIS):.....	15
Version 1	15
Version 2.....	15

The GitHub repository is available here: <https://github.com/zosodk/SynopsisSI>

It is created as a collaborative project for two courses: Databases for Developers and System Integration.

This repository will also serve as a tool for the practical examination of the Distributed Large Systems course, which requires practical examples of the technologies covered in the course.

Introduction

SynopsisSI Project Alignment with Presentation Topics (Detailed & Extensive)

Here's how SynopsisSI e-commerce platform can be leveraged for each of the 12 presentation topics for "Distributed Large Systems" course, with extensive connections:

1. Principles for Large Systems

Define the concept: Start with a general definition of architectural principles for large, distributed systems (e.g., scalability, resilience, maintainability, loose coupling, high cohesion, design for failure, automation).

SynopsisSI - Practical Application:

Loose Coupling:

- Demonstrate: Explain how the API Gateway (YARP) decouples clients from the internal microservice structure.
- Demonstrate: Show how event-driven communication via RabbitMQ/MassTransit (e.g., OrderService publishing OrderPlacedEvent, ListingService consuming it) decouples these services. They don't need direct knowledge or synchronous dependencies for this interaction.

High Cohesion:

- Demonstrate: Each microservice (ListingService, OrderService, UserService) has a well-defined, focused responsibility (its bounded context). ListingService only cares about listings, OrderService only about orders, etc.

Scalability:

- Demonstrate: Explain how each microservice and the API Gateway can be scaled independently (X-axis) by running multiple container instances (refer to docker-compose.yml and the concept of replicas in Kubernetes). Discuss how database choices (MongoDB replica sets/sharding, PostgreSQL read replicas) also support this.

Resilience & Fault Isolation:

- Demonstrate: An error or high load in ListingService should not directly crash OrderService or UserService. Asynchronous messaging with RabbitMQ further enhances this (if ListingService is down, OrderPlacedEvents queue up).
- Demonstrate: Health checks configured in YARP for downstream clusters mean the gateway can stop routing traffic to an unhealthy service instance.

Design for Failure (Conceptual & Implemented Basics):

- Health checks are a basic step.
- Discuss planned resilience patterns like retries and circuit breakers (e.g., with Polly) for inter-service calls.

Automation:

- Demonstrate: Your Dockerfiles automate image creation. Your docker-compose.yml automates local environment setup and service orchestration. Discuss this as a foundation for CI/CD.
- Potential Benefits Demonstrated: Independent scaling, improved fault isolation leading to higher availability, easier maintenance of smaller codebases, ability to choose tech per service.
- Potential Pitfalls Discussed: Increased operational complexity, challenges of distributed transactions (leading to Sagas), inter-service communication overhead, testing complexity.

2. Optimization

SynopsisSI - Practical Application:

Caching with Redis:

- Demonstrate: Explain your plan to implement Redis caching in ListingService for GetListingByIdQuery or for results of SearchListingsQuery. Show where ICacheService would be injected and used.
- Discuss cache invalidation strategies (e.g., when a listing is updated or an OrderPlacedEvent changes its status).

Asynchronous Processing:

- Demonstrate: The OrderService publishing an OrderPlacedEvent which ListingService consumes asynchronously. This optimizes the user-facing order placement API by not waiting for downstream updates.

Database Indexing:

- Demonstrate: Show the OnModelCreating in ListingServiceDbContext (MongoDB) and UserServiceDbContext (PostgreSQL) where indexes are defined (e.g., on SellerId, Category, Email, Username). Explain why these specific indexes optimize common queries.

CQRS (Command Query Responsibility Segregation):

- Demonstrate: Explain that within each service (e.g., ListingService), commands (CreateListingCommand) operate on domain entities for writes, while queries (GetListingByIdQuery) might return tailored DTOs (ListingItemDto) optimized for specific read use cases, preventing over-fetching.

Polyglot Persistence:

- Argue that choosing MongoDB for ListingService (flexible schema, geospatial) and PostgreSQL for UserService (structured identity, relational needs for roles) is an optimization, using the best tool for the job.
- Architectural Principles: Relate these to principles like "Optimize for Read" or "Optimize for Write" where appropriate (CQRS), and "Design for Speed."

3. Design to be Monitored

SynopsisSI - Practical Application:

Structured Logging with Serilog:

- Demonstrate: Your Serilog setup in the Program.cs of ListingService.API, OrderService.API, UserService.API, and SynopsisSI.Gateway.
- Show appsettings.json Serilog configuration for sinks (Console, File) and enrichers (ApplicationName, EnvironmentName, CorrelationId from request logging).
- Show an example of an ILogger<T> being used in a handler with structured properties (e.g., _logger.LogInformation("Processing order {OrderId} for user {UserId}", order.Id, order.UserId);).

Health Check Endpoints:

- Demonstrate: The /health/listings, /health/orders, /health/users endpoints implemented in each service's Program.cs.
- Show the YARP configuration in SynopsisSI.Gateway/appsettings.json that defines how the gateway uses these health checks for its backend clusters.

Distributed Tracing (Planned - OpenTelemetry):

- Explain your detailed plan to integrate OpenTelemetry into Program.cs of all services and the gateway.
- Discuss AddAspNetCoreInstrumentation(), AddHttpClientInstrumentation(), and (conceptually) AddMassTransitInstrumentation() to automatically generate traces.
- Explain how trace context (W3C traceparent) would propagate across HTTP calls (via Gateway) and RabbitMQ messages.
- Mention exporting to Jaeger (runnable in Docker) for visualization.

Metrics (Planned - OpenTelemetry):

- Discuss the plan to collect standard ASP.NET Core metrics, HttpClient metrics, and potentially custom business metrics (e.g., orders placed per minute) using OpenTelemetry.
- Mention exporting to Prometheus and visualizing in Grafana (both runnable in Docker).
- Types of Monitoring: Relate your implementations/plans to application performance monitoring (APM), log aggregation, and infrastructure monitoring.

4. Scalability Cube

SynopsisSI - Practical Application:

X-axis (Horizontal Duplication/Cloning):

- Demonstrate: Explain how each of your .NET microservice APIs (ListingService.API, OrderService.API, UserService.API, SynopsisSI.Gateway) are designed to be stateless and can be scaled by running multiple instances (containers). Refer to the replicas directive in a conceptual Kubernetes deployment or how docker-compose could scale a service (though it's not its primary strength).
- Databases: MongoDB replica sets provide X-axis scaling for reads and HA. PostgreSQL read replicas do the same.
- RabbitMQ/Redis: Can be clustered, which involves X-axis replication for parts of the cluster.

Y-axis (Functional Decomposition/Services):

- Demonstrate: Your entire SynopsisSI architecture is an example of Y-axis scaling. Present your system diagram showing ListingService, OrderService, UserService, and SynopsisSI.Gateway as separate, deployable units, each handling a distinct business function. Contrast this with a conceptual monolith.

Z-axis (Data Partitioning/Sharding):

- MongoDB: Explain how the ListingService or OrderService database could be sharded by listingId, orderId, region, or customerId if it grew very large.
- PostgreSQL: Briefly mention that PostgreSQL also supports sharding (e.g., via table partitioning or extensions like CitusDB), which could be applied to UserService if it had an extreme number of users.
- Redis Cluster: Explain this for distributed caching.

Demonstrate scaling on two axes:

- Y-axis: Your project already demonstrates this by having multiple services.
- X-axis: Show how you would configure docker-compose.yml or (better) Kubernetes to run multiple replicas of ListingService.API. You could even do a simple docker-compose scale listingservice.api=2 as a local demo if you configure a basic load balancer in front (e.g., Nginx in another container, or YARP itself can load balance if a cluster has multiple destinations).

5. Asynchronous Design

SynopsisSI - Practical Application:

Asynchronous

Primary Example: The OrderService publishing OrderPlacedEvent to RabbitMQ, and ListingService consuming it asynchronously via MassTransit.

Demonstrate:

- The OrderPlacedEvent DTO in SynopsisSI.Shared.Events.
- The IMessageBusPublisher interface and its MassTransitMessageBusPublisher implementation in OrderService.Infrastructure.
- The PlaceOrderCommandHandler in OrderService.Application calling _messageBusPublisher.PublishAsync(...).
- The OrderPlacedEventConsumer in ListingService.Application.
- The MassTransit setup in Program.cs of both OrderService.API (for publishing) and ListingService.API (for consumer/receive endpoint).

Contrast with Synchronous:

- Explain what would happen if OrderService directly called ListingService via HTTP to update status:
- OrderService would be blocked waiting for ListingService.
- If ListingService is slow or down, the entire order placement fails.

Tight coupling between the services.

- Pros of Asynchronous (shown by SynopsisSI): Decoupling, increased resilience, improved responsiveness of the OrderService during order placement, independent scalability of the ListingService consumer.
- Cons of Asynchronous: Eventual consistency (listing status update is not in the same transaction as order creation), added complexity of a message broker, need for monitoring queues, potential for message ordering issues (if not handled), requirement for idempotent consumers.

6. Tools for Large Systems (Pick one not used in class)

Choose a tool from SynopsisSI:

YARP (API Gateway):

- Demonstrate: Show its configuration in SynopsisSI.Gateway/appsettings.json (routes, clusters, health checks). Explain how it directs traffic. Show Program.cs setup.
- Pros: .NET native, highly configurable via JSON/code, performant, extensible with ASP.NET Core middleware, good for routing and centralizing some cross-cutting concerns.
- Cons: It's a toolkit, not a full API Management solution (lacks built-in developer portal, advanced analytics, monetization features found in products like Kong or Apigee). Requires .NET development expertise for advanced customization.
- Scalability Support: Can be run in multiple instances behind a load balancer. Efficiently routes to scaled backend services.

MassTransit (Messaging Abstraction):

- Demonstrate: Show the publisher setup in OrderService.API/Program.cs and MassTransitMessageBusPublisher.cs. Show the consumer setup in ListingService.API/Program.cs and OrderPlacedEventConsumer.cs.
- Pros: Simplifies .NET interaction with message brokers (RabbitMQ here), provides abstractions for publish/subscribe, request/response over messaging, sagas, retries, error handling (dead-letter queues). Promotes cleaner application code.
- Cons: Adds another layer of abstraction (learning curve), introduces a dependency on the MassTransit library.
- Scalability Support: Works with scalable message brokers like RabbitMQ (which can be clustered). Allows consumers (your microservices) to be scaled independently to process messages from queues.

Docker Compose (Development Orchestration):

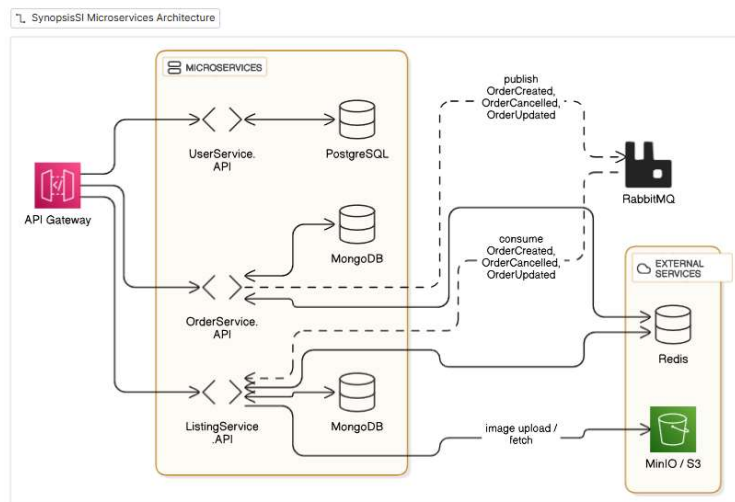
- Demonstrate: Your docker-compose.yml file. Explain how it defines services, networks, volumes, environment variables, and dependencies for the entire SynopsisSI local development stack.
- Pros: Simplifies local setup of complex multi-container applications, ensures consistent development environments, good for integration testing.
- Cons: Not typically a production orchestration tool (Kubernetes is preferred for that); can become complex for very large numbers of services.
- Scalability Support: While docker-compose itself doesn't scale applications in production, it allows you to develop and test scalable architectures locally by simulating multiple services. It uses Docker, which is fundamental to most scalable containerized deployments.

7. Fault-isolative Architectures

SynopsisSI - Practical Application:

Microservice Boundaries:

- Demonstrate: Use your system diagram. Explain that ListingService, OrderService, and UserService are separate processes (containers). A crash or resource exhaustion in one (e.g., a memory leak in ListingService) will not directly crash the others.



Asynchronous Communication via RabbitMQ:

- Demonstrate: If ListingService is down, OrderService can still accept orders and publish OrderPlacedEvent messages to RabbitMQ. These messages will be queued and processed by ListingService once it recovers. This isolates the order capture functionality from ListingService's availability.

API Gateway (YARP) with Health Checks:

- Demonstrate: Show the health check configuration in SynopsisSI.Gateway/appsettings.json for the service clusters. Explain that YARP can detect an unhealthy instance of a service (e.g., listingservice.api) and stop routing traffic to it, preventing users from hitting a failing service.

Database per Service:

- A failure in the MongoDB instance used by ListingService will not directly affect the PostgreSQL database used by UserService.

(Conceptual/Planned) Circuit Breaker Pattern (Polly):

- Explain how you would add Polly's Circuit Breaker to any synchronous HTTP calls made between services (e.g., if OrderService needed to synchronously fetch product price from ListingService before creating an order). This would prevent the calling service from repeatedly hammering a failing downstream service.
- Purpose & Benefits: Discuss how these design choices in SynopsisSI reduce the "blast radius" of failures, improve overall system uptime and user experience by allowing graceful degradation of non-critical functionalities.

8. Specifications and Outsourcing

SynopsisSI - Practical Application:

API Contracts (OpenAPI/Swagger):

- Demonstrate: Show the Swagger UI for ListingService.API (or another service). Explain that the generated OpenAPI specification (JSON/YAML) is a formal contract describing its endpoints, request/response schemas, and authentication requirements.

Event Contracts (SynopsisSI.Shared.Events):

- Demonstrate: Show the C# DTOs like OrderPlacedEvent.cs and UserRegisteredEvent.cs. These define the schema for messages passed between services via RabbitMQ.

Application Layer Interfaces:

- Demonstrate: Show interfaces like IListingRepository.cs or IUnitOfWork.cs from ListingService.Application. These define the contracts that the Infrastructure layer must implement.

How to use for outsourcing:

- Explain that if you wanted to outsource the development of a new, independent ReviewService, you would provide:
- The API contract for how the ReviewService should expose its functionality (e.g., POST /api/reviews, GET /api/listings/{listingId}/reviews).
- The event contracts for any events it needs to consume (e.g., an OrderCompletedEvent to know when a review can be submitted) or publish (e.g., ReviewSubmittedEvent).
- The outsourced team could then develop the ReviewService in isolation, building against these contracts, without needing access to the internal codebase of other SynopsisSI services. This demonstrates clear component specification enabling distributed/outsourced development.

9. Feature Flags

SynopsisSI - Practical Application (Conceptual/Planned):

Scenario: Imagine introducing a new

"Premium Listing"

feature in ListingService that allows sellers to pay for better visibility. This feature might involve new UI elements, API changes, and backend logic.

Demonstrate (Conceptually):

Explain how you would use a library like Microsoft.FeatureManagement in ListingService.API and ListingService.Application.

Show a conceptual code snippet in ListingsController.cs or a handler:

```
C#
// if (await _featureManager.IsEnabledAsync("PremiumListings")) {
//     // Execute premium listing logic
// } else {
//     // Execute standard listing logic
// }
```

The feature flag ("PremiumListings") would be configured in appsettings.json or, ideally, a dynamic feature flag management service (like LaunchDarkly, Azure App Configuration Feature Flags).

Pros & Cons:

- **Pros:** Decouple deployment from release (deploy code with the feature off, then turn it on), A/B testing, canary releases, gradual rollouts, dark launching (testing in production with internal users), quickly disable a problematic feature.
- **Cons:** Increases code complexity (more conditional logic), testing all combinations of flags can be challenging, "dead" flagged code can accumulate if not managed.
- **DevOps & Scalability:** Directly relates to DevOps principles of faster, safer releases. Allows testing new, potentially resource-intensive features with a subset of users or traffic before a full-scale rollout, helping to understand scalability impact.

10. Availability

SynopsisSI - Practical Application:

Microservice Independence & Redundancy (X-axis Scaling):

- Demonstrate: Explain how each critical microservice (SynopsisSI.Gateway, ListingService.API, OrderService.API, UserService.API) is designed as a separate container. Discuss how, in a production environment (e.g., Kubernetes), you would configure multiple replicas for each of these services. If one instance fails, the load balancer routes traffic to healthy instances.
- Similarly, discuss how MongoDB and PostgreSQL replica sets provide database availability.

Asynchronous Communication via RabbitMQ:

- Demonstrate: If ListingService (the consumer of OrderPlacedEvent) is temporarily unavailable, RabbitMQ queues the messages. OrderService (the publisher) can continue to function and accept orders. Once ListingService recovers, it processes the backlog of events, ensuring no data is lost and improving the perceived availability of the order placement feature.

API Gateway Health Checks:

- Demonstrate: Show the YARP health check configuration in SynopsisSI.Gateway/appsettings.json. Explain that YARP actively monitors downstream services and will avoid routing requests to instances that are unhealthy, preventing users from hitting failing services.

Fault Isolation by Design:

- The inherent boundaries between microservices mean that a critical bug or resource exhaustion in, for example, a feature (non-critical) RecommendationService would not directly bring down core services like OrderService or UserService.

How you could potentially harm availability:

- Introducing tight synchronous dependencies between critical path services.
- Having single points of failure in shared infrastructure (e.g., a non-clustered message broker or a single database instance without replicas for a critical service).
- Long timeouts for inter-service calls without proper circuit breakers.
- Deploying all microservices simultaneously in a "big bang" release.
- Relation to Architectural Principles & Scale Cube: Relate this to "Design for Failure." X-axis and Y-axis scaling directly contribute to higher availability.

11. DevOps

SynopsisSI - Practical Application (Foundational & Planned):

Automation:

Build & Containerization:

- Demonstrate your Dockerfile for ListingService.API (or another service) and the docker-compose.yml. Explain how these automate the creation of consistent, reproducible images and local development environments. This is the first step in an automated pipeline.

Testing (Planned):

Discuss your strategy for automating different test types in a CI pipeline:

- Unit tests within each service's layers (dotnet test).
- Integration tests for each service interacting with its database (e.g., using Testcontainers or a test DB instance).
- Contract tests (e.g., using Pact) to verify API and event contracts between services.
- End-to-end tests (via the API Gateway).

Deployment (Conceptual for CI/CD):

- Explain how the Docker images built for SynopsisSI services would be pushed to a container registry (e.g., Docker Hub, Azure Container Registry, AWS ECR).
- Discuss how a CI/CD pipeline (e.g., GitHub Actions, GitLab CI, Jenkins) would orchestrate the build, test, and deployment of these images to different environments (Dev, Staging, Production), potentially using Kubernetes and Helm for production deployments.
- Mention deployment strategies like blue/green or canary releases, which are facilitated by microservices and good CI/CD.

Recovery:

- Microservices allow for faster, independent rollbacks of a single faulty service if a deployment goes wrong.
- Container orchestration platforms (like Kubernetes, which Docker Compose can be a precursor to) provide self-healing by automatically restarting failed container instances.
- Your health checks contribute to this by allowing the orchestrator to detect failures.
- Relate to Architectural Principles: Microservice architecture itself supports DevOps by enabling smaller, independent deployments (reducing risk and complexity), faster feedback loops, and team autonomy.

12. Green Software

SynopsisSI - Practical Application & Considerations:

Resource Efficiency of Microservices (Y-axis & X-axis scaling):

- Argue that the ability to scale individual microservices (ListingService, OrderService, etc.) independently based on their actual load is more energy-efficient than scaling a large monolith. In a monolith, all components are scaled together, meaning even underutilized parts consume resources (CPU, memory) when the overall application is scaled up for peak load in one area. With microservices, you can allocate resources (e.g., smaller/fewer container instances) more precisely to where they are needed.

Optimized Database Choice (Polyglot Persistence):

- Explain that using MongoDB for the ListingService and OrderService (where its document model and query patterns can be very efficient for handling flexible, nested data) and PostgreSQL for the UserService (leveraging relational strengths for structured identity data) allows each service to use a data store optimized for its specific workload. This can lead to faster operations and less wasted compute per transaction/query compared to forcing all data into a single, sub-optimal database type.

Asynchronous Processing & Event-Driven Architecture:

- The use of RabbitMQ for tasks like updating listing status after an order is placed means these operations don't block the primary user flow. This can smooth out processing load, reduce resource contention during peak user interactions, and potentially allow some background workloads to be deferred or run on compute resources that are scaled more efficiently (or even scaled down during off-peak hours).

Caching (Redis):

- By caching frequently accessed data (e.g., popular listings, user profile snippets) in Redis, SynopsisSI reduces the number of direct queries to its primary databases (MongoDB, PostgreSQL). This lessens the load on database servers, reducing their energy consumption and often leading to faster response times for users.

Demonstration (Conceptual Comparison for SynopsisSI):

Version 1

(Less Green - Monolithic Analogy or Less Optimal Microservice Interaction):

- Imagine the "Place Order" process required the OrderService to make multiple, sequential, synchronous HTTP calls to ListingService (to check and update stock for each item), then to UserService (to verify buyer details not already cached), and then to a NotificationService, all while the user waits. If any of these services are slow, the entire process holds open connections and consumes resources for longer, leading to higher energy use per transaction. If the monolith had to scale for peak order processing, all other less-used modules would also scale.

Version 2

(More Green - SynopsisSI's Event-Driven & Scaled Approach):

- The OrderService quickly handles the core order creation and then publishes an OrderPlacedEvent. The ListingService and a future NotificationService consume this event asynchronously. This means the initial user interaction is fast, and resource usage for that interaction is minimized. The event consumers can process their tasks independently and can be scaled based on the queue length, rather than being tied to the synchronous order placement flow. Each service (ListingService, OrderService, UserService) can be scaled based on its individual demand, leading to more efficient overall resource (and thus energy) consumption.
- Why important for large systems: The cumulative energy footprint of large, distributed systems can be substantial. Applying green software principles (like demand shaping through async processing, resource efficiency via independent scaling, and data-access optimization through polyglot persistence) can lead to significant reductions in energy use, operational costs, and environmental impact.